L16 — Linear Linked List: Representation in Memory

Unit: 2 | **Chapter:** Linked Lists | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Explain how a linked list is stored in memory using nodes and pointers
- Compare linked list and array memory models
- Implement a node structure and a basic linked list
- Describe the advantages and limitations of linked lists

---

1. Limitations of Arrays — Motivation for Linked Lists

Array Problem	Impact
Fixed size at compile time	Cannot grow dynamically
Insertion at index i costs O(n)	Shifting required
Deletion at index i costs O(n)	Shifting required
Contiguous memory required	May fail if no large block available

Linked lists solve these by using dynamic memory — each element can live anywhere in memory.

---

2. Linked List Structure

A **linked list** is a linear data structure where each element (called a **node**) contains:

1. **Data field** — stores the actual value
2. **Link field (pointer/reference)** — stores the address of the next node

```
Node structure:
┌──────────┬──────────┐
│  DATA    │  NEXT →  │
└──────────┴──────────┘
```

```
A linked list of 4 nodes:
HEAD
  ↓
[10 | 200] → [20 | 300] → [30 | 400] → [40 | NULL]
  @100         @200         @300         @400
```

The last node's `NEXT` is `NULL` (or `None` in Python), marking the end.

---

3. Memory Representation

Unlike arrays, linked list nodes are stored at **non-contiguous** memory locations:

```
Memory Address | Content
──────────────────────────
100            | data=10, next=200   ← HEAD points here
150            | (some other variable, not part of list)
200            | data=20, next=300
300            | data=30, next=400
400            | data=40, next=NULL
```

The list is connected through pointers regardless of physical memory positions.

---

4. Node Implementation

In Python

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None    # Initially points to nothing
```

In C

```
struct Node {
    int data;
    struct Node *next;   // Self-referential structure
};
```

---

5. Linked List Class

```
class LinkedList:
    def __init__(self):
        self.head = None    # Initially empty list

    def is_empty(self):
        return self.head is None

    def display(self):
        current = self.head
        elements = []
        while current:
            elements.append(str(current.data))
            current = current.next
        print(" → ".join(elements) + " → NULL")

    def length(self):
        count = 0
```

```
        current = self.head
        while current:
            count += 1
            current = current.next
        return count  # O(n)
```

---

6. Types of Linked Lists

Type	Description	Last Node Points To
Singly Linked List	Each node points to next only	NULL
Doubly Linked List	Each node has prev and next pointers	NULL
Circular Linked List	Last node points back to head	HEAD
Circular Doubly	Both directions, last connects to first	HEAD

```
Singly:   HEAD → [10|→] → [20|→] → [30|NULL]

Doubly:   HEAD ← [10|→] ↔ [20|→] ↔ [30|NULL]
          NULL←|     |     |     |     |     |→NULL

Circular: HEAD → [10|→] → [20|→] → [30|→] ↗ (points back to HEAD)
```

---

7. Linked List vs Array

Property	Array	Linked List
Memory	Contiguous	Non-contiguous
Size	Fixed	Dynamic
Access by index	O(1)	O(n)
Insert at beginning	O(n)	**O(1)**
Insert at end	O(1) (if tracking end)	O(n) or O(1) with tail pointer
Delete from beginning	O(n)	**O(1)**
Memory overhead	None	Extra pointer per node
Cache performance	Excellent	Poor

---

8. Creating a Linked List

```
# Building: 10 → 20 → 30 → 40 → NULL

ll = LinkedList()

# Method 1: Manual construction
n1 = Node(10)
n2 = Node(20)
n3 = Node(30)
n4 = Node(40)
```

```
n1.next = n2
n2.next = n3
n3.next = n4

ll.head = n1
ll.display()   # 10 → 20 → 30 → 40 → NULL
```

---

9. Head and Tail Pointers

```
class LinkedListWithTail:
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0

    def append(self, data):
        """Add to end: O(1) with tail pointer"""
        new_node = Node(data)
        if self.tail:
            self.tail.next = new_node
        else:
            self.head = new_node
        self.tail = new_node
        self.size += 1
```

With a **tail pointer**, appending to end becomes O(1) instead of O(n).

---

10. Memory Overhead

Each node requires:

- Data: e.g., 4 bytes (int)
- Pointer: 8 bytes (64-bit system)

Overhead: $8/4 = 200\%$ pointer overhead for integer data.

For larger records (e.g., 100-byte records), pointer overhead is only 8%: $$\text{Overhead} = \frac{8}{100} = 8\%$$

This is why linked lists are preferred when nodes store large records.

Summary

- A **linked list** is a dynamic collection of nodes connected by pointers.
- Nodes are stored at **non-contiguous** memory locations — no shifting required for insertion/deletion.
- Singly linked: each node has `data` and `next`.
- Linked list trades **random access (O(n))** for **O(1) insertion/deletion at known positions**.

- Extra pointer per node adds memory overhead — larger data makes this negligible.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 10.2 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.1 (Springer, 2020)